



Ściąga egzaminacyjna

WisentPedigree DataCleaner v1.2.0

21 testów passed

projekt do prezentacji: integracja danych EBPB + aplikacja Streamlit + CLI

UI / CLI

Loader danych

Schemat pandas

Walidacja

Auto-fix / ręczne

Eksport

1. O projekcie

Jedno zdanie: aplikacja przygotowuje bazy rodowodowe żubrów EBPB do dalszej analizy: importuje, standaryzuje, waliduje, czyści i eksportuje dane z raportem.

- Problem: różne eksporty EBPB, niespójne ID, rodzice, płeć i daty.
- Cel: powtarzalne i udokumentowane przygotowanie danych.
- Zakres: kontrola jakości danych, nie pełna platforma hodowlana.

2. Wymagane części projektu

Część integracyjna

- import CSV/XLS/XLSX
- rozpoznanie raportu i rejestru EBPB
- wspólny schemat pandas
- łączenie baz, walidacja, eksport

Część główna

- aplikacja Streamlit
- pięć kroków pracy
- ten sam silnik dostępny przez CLI

3. Kolejność w aplikacji

- Krok 1 - Wczytanie danych
- Krok 2 - Walidacja
- Krok 3 - Czyszczenie automatyczne
- Krok 4 - Czyszczenie ręczne
- Krok 5 - Wyniki

```
cd zubry_pedigree_app && python run_cli.py run --input data/EBPB_bison_report.xlsx --project-name terminal_demo
```

4. Struktura aplikacji

app/data	-> import, schemat, walidacja, auto-fix
app/huba	-> silnik procesu i eksport
app/ui/streamlit	-> interfejs użytkownika
app/pedigree	-> osoby i relacje rodzic-potomstwo
app/analytics	-> metryki pokrewieństwa i inbredu
config	-> przykładowe JSON
data	-> przykładowe dane EBPB
tests	-> testy jakości

5. Co sprawdza walidacja

- brak ID i duplikaty ID
- nieznane kody płci
- brakujące rekordy rodziców
- self-parent i cykle w grafie
- niezgodność płci z rolą rodzica
- zakres lat, daty i wiek rodzica
- linie rodowodowe

6. Wyniki i artefakty

- oczyszczone CSV/XLSX
- comparison.csv - przed i po
- final_report.html - raport końcowy
- manifest.json - wersja, konfiguracja, SHA-256
- ZIP z katalogiem wyników

7. Pliki do pokazania

- README.md - instalacja, diagram, AI
- PRZEWODNIK_EGZAMINACYJNY.md
- app/data/dataset_loader.py
- app/data/validator.py
- app/data/auto_fix.py
- app/huba/engine.py
- app/ui/streamlit/streamlit_app.py
- tests/ - potwierdzenie jakości

8. Gotowa odpowiedź o AI

AI, w tym Codex, było wsparciem programistycznym: porządkowanie kodu, dokumentacji, testów i przegląd spójności. AI nie działa w aplikacji, nie przetwarza danych użytkownika i nie podejmuje decyzji hodowlanych.

9. Ograniczenia i finał

- aplikacja wspiera eksperta, nie zastępuje go
- auto-fix nie odtwarza informacji biologicznej
- wynik wymaga oceny domenowej

Zdanie końcowe:

Projekt rozwiązuje konkretny problem: powtarzalne przygotowanie danych rodowodowych żubrów do dalszej analizy.